

1 Multiplicative algebra on the completed carrier

Source: PrimeTensor/Algebra.lean

What this file establishes

PrimeTensor/Completion.lean produced the carrier MulReal as a quotient of Cauchy streams and left it with no operations at all. This file supplies them. Multiplication and inversion are lifted from MulRat to streams and then through asymptotic equivalence, and the resulting MulReal is shown to be a commutative group: associative, commutative, unital, and with $a \cdot a^{-1} = 1$ for every element.

The lift runs in three strata, and it is worth naming them because the work is very unevenly distributed between them.

- (i) *The operations respect the scale.* Two theorems, `scaleWithin_mul` and `scaleWithin_inv`. This is the whole analytic content of the file.
- (ii) *Hence they act on streams and respect asymptotic equivalence.* Four routine consequences, all of them the same anchor-juggling already seen in the transitivity proof of PrimeTensor/Completion.lean.
- (iii) *Hence they descend to the quotient, and the group laws hold there.* Seven laws, none of which involves an anchor or a level at all: each holds *termwise* on streams, and of `_pointwise` converts a termwise equality into an equality of classes.

So the analysis is confined to stratum (i), two theorems, and the algebra of stratum (iii) is in every case “the same law one layer down”. What makes that separation possible is that the group laws are exact identities on MulRat, so no error term survives to be estimated.

1.1 Ratios of products and of inverses

Lemma 1.1 (`MulRat.inv_mul_pair` and `MulRat.inv_one`). $(a \cdot b)^{-1} = a^{-1} \cdot b^{-1}$, and $1^{-1} = 1$.

```
theorem inv_mul_pair (a b : MulRat) :  
  (a * b)-1 = a-1 * b-1 := by  
  refine Quotient.inductionOn₂ a b ?_  
  intro x y  
  change  
  Quotient.mk PrimeRatio.setoid ((x * y)-1) =  
    Quotient.mk PrimeRatio.setoid (x-1 * y-1)  
  rfl
```

Proof. Both are `rfl` once representatives are chosen. On PrimeRatio, multiplication is componentwise and inversion exchanges the two components, so $(xy)^{-1}$ and $x^{-1}y^{-1}$ are the same pair $\langle x.lower \cdot y.lower, x.upper \cdot y.upper \rangle$ — not merely equivalent modulo the setoid, but syntactically identical. Likewise $1 = \langle 1, 1 \rangle$ is fixed by the exchange. $\square$

Lemma 1.2 (`MulRat.mul_four_shuffle`, private). $(ab)(cd) = (ac)(bd)$.

Proof. Five rewrites, alternating associativity and the single commutation $bc = cb$. As in `PrimeTensor/Scale/Laws.lean`, the regrouping is written out because `MulRat` carries `Mul` and `One` instances but no `CommMonoid`, so `ac_rfl` is not available. $\square$

Lemma 1.3 (`MulRat.ratio_mul_pair`). $\text{ratio}(ac, bd) = \text{ratio}(a, b) \cdot \text{ratio}(c, d)$.

Proof. Unfolding `ratio`, the left side is $(ac)(bd)^{-1}$, which Lemma 1.1 turns into $(ac)(b^{-1}d^{-1})$, and Lemma 1.2 regroups into $(ab^{-1})(cd^{-1})$. $\square$

Design note 1.4 (The multiplicative product rule is exact). Lemma 1.3 is the statement that *the relative error of a product is the product of the relative errors*. It is an identity, with no remainder and no cross term.

Its additive counterpart is not. Estimating $|ac - bd|$ requires splitting $ac - bd = a(c - d) + (a - b)d$ and then *bounding a and d* , which is why the classical construction of $\mathbb{R}$ from Cauchy sequences has to prove first that a Cauchy sequence is bounded, and why the proof that a product of Cauchy sequences is Cauchy cites that lemma. Nothing of the sort appears in this file, or anywhere in this development: there is no boundedness lemma because there is no cross term to bound. The multiplicative presentation buys this by measuring error as a ratio rather than a difference, and a ratio is multiplicative on the nose.

Lemma 1.5 (`MulRat.ratio_inv_pair`). $\text{ratio}(a^{-1}, b^{-1}) = \text{ratio}(b, a)$.

Proof. $\text{ratio}(a^{-1}, b^{-1}) = a^{-1}(b^{-1})^{-1} = a^{-1}b = ba^{-1} = \text{ratio}(b, a)$, using `inv_inv` and commutativity from `PrimeTensor/MulRat.lean`. $\square$

1.2 The two analytic theorems

Theorem 1.6 (`MulRat.scaleWithin_mul`). *If*

$$\text{ScaleWithin}(\text{succ } \ell, a, b) \quad \text{and} \quad \text{ScaleWithin}(\text{succ } \ell, c, d),$$

then $\text{ScaleWithin}(\ell, a \cdot c, b \cdot d)$.

Proof. Identical in shape to `scaleWithin_comp` of `PrimeTensor/Scale/Laws.lean`, with Lemma 1.3 in place of the composition of ratios. Unfolding `scalePow` at `succ  $\ell$` turns the four hypotheses into squares below two. For the forward conjunct, rewrite $\text{ratio}(ac, bd) = \text{ratio}(a, b)\text{ratio}(c, d)$, distribute the ladder over that product by `scalePow_mul`, and apply `mul_lt_two_of_sq_lt_two`. For the backward conjunct, rewrite instead $\text{ratio}(bd, ac) = \text{ratio}(b, a)\text{ratio}(d, c)$ and use the two backward hypotheses. $\square$

Theorem 1.7 (`MulRat.scaleWithin_inv`). $\text{ScaleWithin}(\ell, a, b)$ *implies* $\text{ScaleWithin}(\ell, a^{-1}, b^{-1})$ — *at the same level*.

```
theorem scaleWithin_inv {level : Depth} {a b : MulRat}
  (h : ScaleWithin level a b) :
  ScaleWithin level a⁻¹ b⁻¹ := by
  unfold ScaleWithin at h ⊢
  rw [ratio_inv_pair a b, ratio_inv_pair b a]
  exact ⟨h.2, h.1⟩
```

Proof. By Lemma 1.5 the two oriented ratios of the inverted pair are the two oriented ratios of the original pair, exchanged. So the goal is the hypothesis with its conjuncts swapped. $\square$

Design note 1.8 (Inversion is an isometry, and why that is free). Theorem 1.7 costs no level: closeness at ℓ goes to closeness at ℓ , and the proof is a swap. Multiplicative distance is literally invariant under $x \mapsto x^{-1}$, which merely reverses orientation, and ScaleWithin was defined symmetrically in the two orientations precisely so that the reversal is invisible.

Contrast the classical construction again. There, showing that the reciprocal of a Cauchy sequence is Cauchy is the delicate step: $x \mapsto 1/x$ is not uniformly continuous near 0, the estimate $|1/a - 1/b| = |a - b|/|ab|$ blows up, and one must first prove that a sequence not asymptotic to 0 is eventually bounded *away* from 0, then restrict to such sequences, then check the choice is independent of representative. None of that appears here, and the reason is structural rather than clever: MulRat is the strictly positive rationals, so the singular point is not in the carrier to begin with. The completion is a group because the space being completed already was one, with no punctured-domain bookkeeping to carry through the quotient.

Remark 1.9 (One level for multiplication, none for inversion). The asymmetry is exactly right. Multiplication combines two independent errors, so their ratios multiply and the level must absorb a squaring — the same single-level cost, for the same reason, as composing two steps through a midpoint. Inversion combines nothing; it is a bijection of the carrier that preserves multiplicative distance, so it is free. Every level bookkeeping in the rest of the file traces back to this one distinction: the constructions that combine two streams take a join of anchors and ask their hypotheses for $\text{succ } \ell$, and the constructions that transform one stream keep both the anchor and the level.

1.3 Operations on streams

Definition 1.10 (MulCauchyStream.mul and MulCauchyStream.inv). The product of two Cauchy streams is the pointwise product, and the inverse is the pointwise inverse; each carries a proof that the result is again Cauchy. The two `simp` lemmas `mul_term` and `inv_term` record that the terms are what they look like, both by `rfl`.

Proof that the product is Cauchy. Given a level ℓ , ask each factor’s Cauchy property for $\text{succ } \ell$, obtaining anchors a_a and a_b , and take $\text{join}(a_a, a_b)$. For m and n in that tail, the two bounds on join and transitivity of `AtOrAfter`, all from `PrimeTensor/Completion.lean`, place both indices in both original tails, so both factors are close at $\text{succ } \ell$, and Theorem 1.6 combines them at ℓ . $\square$

Proof that the inverse is Cauchy. Ask the Cauchy property for ℓ itself, keep the same anchor, and apply Theorem 1.7 termwise. No join and no `succ` occur. $\square$

1.4 Compatibility with asymptotic equivalence

Lemma 1.11 (MulAsymptotic.of_pointwise). *If $a_n = b_n$ for every n , then a and b are asymptotic.*

```
theorem of_pointwise {a b : MulCauchyStream}
  (h : ∀ n : Depth, a.term n = b.term n) :
  MulAsymptotic a b := by
  intro level
  refine (.one, ?_)
  intro n hn
  rw [h n]
  exact MulRat.scaleWithin_refl level (b.term n)
```

Proof. Take the anchor one, so the tail is everything, rewrite by the pointwise equality, and apply unconditional reflexivity of ScaleWithin. $\square$

This small lemma is what makes stratum (iii) cheap. Every group law below is an equality that already holds term by term, so none of them needs a level, an anchor or an estimate: of_pointwise turns the termwise identity straight into the required asymptotic equivalence, and Quotient.sound turns that into an equality in MulReal.

Lemma 1.12 (MulAsymptotic.mul and MulAsymptotic.inv). *MulAsymptotic is a congruence for both operations: from $a \sim a'$ and $b \sim b'$ follows $ab \sim a'b'$, and from $a \sim b$ follows $a^{-1} \sim b^{-1}$.*

Proof. For the product, the same three moves as the transitivity proof of PrimeTensor/Completion.lean: ask both hypotheses for succ ℓ , take join of the anchors, and combine with Theorem 1.6. For the inverse, keep the anchor and the level and apply Theorem 1.7 termwise. $\square$

1.5 The operations on MulReal

Definition 1.13 (MulReal.mul and MulReal.inv). mul is Quotient.liftOn₂ of the stream product and inv is Quotient.liftOn of the stream inverse, each with Lemma 1.12 as its well-definedness obligation. They are registered as the Mul and Inv instances of MulReal.

```
def inv (a : MulReal) : MulReal :=
  Quotient.liftOn a
    (fun x => ofStream (MulCauchyStream.inv x))
  (by
    intro a b h
    change MulAsymptotic a b at h
    change
      Quotient.mk MulAsymptotic.setoid (MulCauchyStream.inv a) =
        Quotient.mk MulAsymptotic.setoid (MulCauchyStream.inv b)
    apply Quotient.sound
    exact MulAsymptotic.inv h)
```

Theorem 1.14 (The group laws). *MulReal is a commutative group under mul with unit $1 = \text{ofRat}(1)$ and inverse inv. Explicitly: $1 \cdot a = a$ and $a \cdot 1 = a$; $(ab)c = a(bc)$; $ab = ba$; $(a^{-1})^{-1} = a$; $a \cdot a^{-1} = 1$ and $a^{-1} \cdot a = 1$.*

Proof. All but the last have the same four-line shape: induct to replace each element by a representative stream, apply Quotient.sound to reduce the claim to an asymptotic equivalence, apply Lemma 1.11 to reduce that to a termwise equality, and close the termwise equality with the corresponding law of PrimeTensor/MulRat.lean — one_mul, mul_one, mul_assoc, mul_comm, inv_inv, mul_inv respectively. The unit cases additionally use that the constant stream at 1 has every term 1. The last law, $a^{-1}a = 1$, is obtained from the previous two by commuting, exactly as in PrimeTensor/Scale/Laws.lean. $\square$

Remark 1.15 (Where the analysis went). Not one of the seven laws mentions a level, an anchor, or ScaleWithin. That is not an accident of presentation: the laws hold strictly, term by term, because they hold in MulRat, and equality of streams is finer than asymptotic equivalence. All of the analytic content of the file was spent in Theorems 1.6 and 1.7, which are what make

the operations well defined on the quotient at all; once they are proved, the group structure descends for free.

The same will not be true of an order or of a limit, where the statement itself mentions the scale and cannot be checked termwise.

Remark 1.16 (Still a bare `Mul`, not a `CommGroup`). The seven laws amount to `MulReal` being an abelian group, but no `CommGroup` instance is declared: only `One`, `Mul` and `Inv`. This repeats the choice made for `MulRat`, and it has the same consequence, visible already in this file — the regrouping of four factors in Lemma 1.2 has to be written by hand rather than discharged by `ac_rfl` or `group`. It keeps the development independent of the library’s algebraic hierarchy, at the price of re-proving inside it whatever the hierarchy would have supplied.

Remark 1.17 (What is still absent). There is no order on `MulReal`, no limit notion, and no statement that `MulReal` is complete for the scale it was built from — the standard next obligation for any completion. Nor is `ofRat` shown to be a homomorphism or shown injective, although the first would now follow from Lemma 1.11 in one line, since the constant stream of a product is the product of the constant streams. Consistently with the module header, nothing additive appears: no addition, no zero, no subtraction, and no coefficient drawn from $\mathbb{Q}$ or $\mathbb{R}$.

Summary of the correspondence

Lean declaration	Kind	Here
<code>MulRat.inv_mul_pair</code>	theorem	Lem. 1.1
<code>MulRat.inv_one</code>	simp thm	Lem. 1.1
<code>MulRat.mul_four_shuffle</code>	private thm	Lem. 1.2
<code>MulRat.ratio_mul_pair</code>	theorem	Lem. 1.3
<code>MulRat.ratio_inv_pair</code>	theorem	Lem. 1.5
<code>MulRat.scaleWithin_mul</code>	theorem	Thm. 1.6
<code>MulRat.scaleWithin_inv</code>	theorem	Thm. 1.7
<code>MulCauchyStream.mul</code>	definition	Def. 1.10
<code>MulCauchyStream.inv</code>	definition	Def. 1.10
<code>MulCauchyStream.mul_term</code>	simp thm	Def. 1.10
<code>MulCauchyStream.inv_term</code>	simp thm	Def. 1.10
<code>MulAsymptotic.of_pointwise</code>	theorem	Lem. 1.11
<code>MulAsymptotic.mul</code>	theorem	Lem. 1.12
<code>MulAsymptotic.inv</code>	theorem	Lem. 1.12
<code>MulReal.mul</code>	definition	Def. 1.13
<code>MulReal.inv</code>	definition	Def. 1.13
<code>Mul MulReal</code>	instance	Def. 1.13
<code>Inv MulReal</code>	instance	Def. 1.13
<code>MulReal.one_mul</code>	simp thm	Thm. 1.14
<code>MulReal.mul_one</code>	simp thm	Thm. 1.14
<code>MulReal.mul_assoc</code>	theorem	Thm. 1.14
<code>MulReal.mul_comm</code>	theorem	Thm. 1.14
<code>MulReal.inv_inv</code>	simp thm	Thm. 1.14
<code>MulReal.mul_inv</code>	simp thm	Thm. 1.14
<code>MulReal.inv_mul</code>	simp thm	Thm. 1.14

Every declaration of `PrimeTensor/Algebra.lean` appears above. The file contains no `sorry`, no axiom, and no unproved named proposition. Remarks 1.9, 1.15, 1.16 and 1.17 are stated for the reader and are not declarations of the file.